WebMCP Scale Test — 150 Products (Real Browser)

Generated 13 Oct 2026 08:23 UTC · brain: glm-5.3-flash (temp 0) · same 8 shopping tasks · catalog 30 → 150 products (5×) · identical stores on demowithmcp / demonomcp.converso.work · all site traffic inside headless Chrome

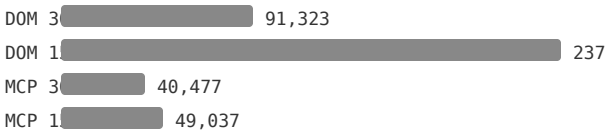
Headline — 150 products

	Success	Action data ↓	Requests	LLM tok in	LLM tok out	Agent time	Wire page bytes
DOM (demonomcp)	8/8	237,183 B	30	68,932	8,681	74.4 s	446,063 B
MCP (demowithmcp)	8/8	**49,037 B (−79%)**	53	**39,754 (−42%)**	2,959	57.3 s	298,657 B

The crossover — what 5× more catalog did to each side

	Bytes @30	Bytes @150	Δ bytes	Tok in @30	Tok in @150	Δ tokens	Time @30	Time @150
DOM	91,323 B	237,183 B	**+160%**	13,603	68,932	**+407%**	25.3 s	74.4 s
MCP	40,477 B	49,037 B	**+21%**	34,796	39,754	**+14%**	74.3 s	57.3 s

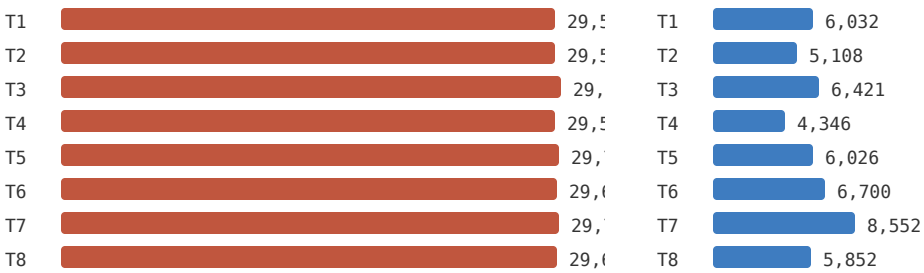
Cost growth 30→150 (action bytes)



DOM grew +160% with the catalog. MCP grew +21%.

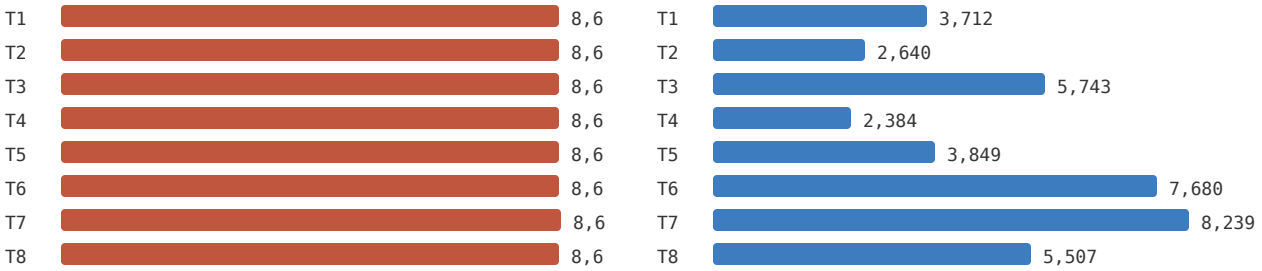
Task-by-task — 150 products

Action bytes ↓



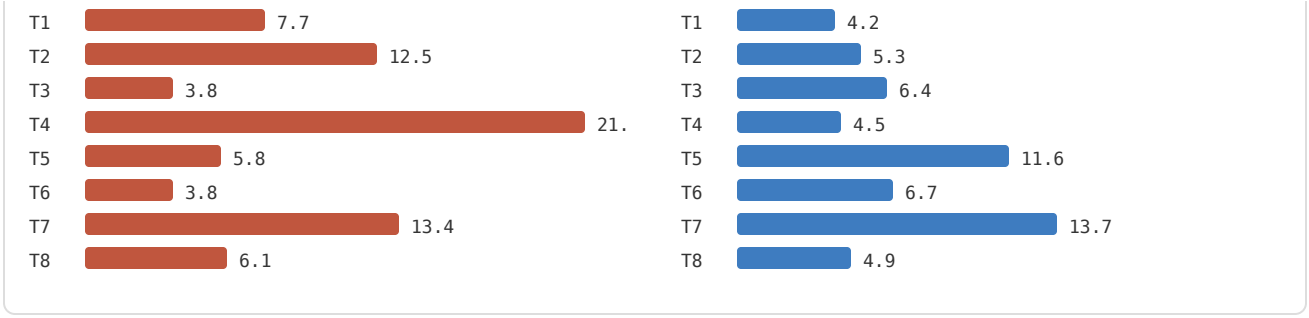
DOM fetches the full 150-product catalog (~29.7KB) every task. MCP fetches task-sized slices (4.3–8.6KB).

LLM input tokens



MCP re-sends tool schemas per turn but search slices are small — it now also wins tokens (−42%)

Wall-clock s



Choices & orders (150-product run)

Task	DOM pick	Order	MCP pick	Order	Same choice?
T1	Logitech Mouse Travel 751	—	Logitech Mouse Travel 751	—	✓ identical
T2	HP Pavilion 14	—	HP Pavilion 14	—	✓ identical
T3	iQOO Z9s Pro	1009	iQOO Z9s Pro	1011	✓ identical
T4	Razer DeathAdder Essential	—	Razer DeathAdder Essential	—	✓ identical
T5	MacBook Air M3 13	1010	MacBook Air M3 13	1012	✓ identical
T6	Google Edge 2 5G	1011	Google Edge 2 5G	1013	✓ identical
T7	Logitech MX Master 3S	1012	Logitech MX Master 3S	1014	✓ identical
T8	OnePlus 12R	1013	OnePlus 12R	1015	✓ identical

All 8 picks on each site = the independent ground-truth answers recomputed against the 150-product catalog (HP Pavilion 14, iQOO Z9s Pro, Razer DeathAdder, MacBook Air M3, OnePlus 12R…). 16/16 correct.

Findings — the crossover landed

- **DOM cost scales with the catalog**: total action bytes 91,323 → 237,183 (+160%) when the catalog grew 5×; every task re-drags the whole catalog (~29.7KB each, flat across tasks).
- **MCP cost scales with the task**: 40,477 → 49,037 (+21%). search_products returns only the matching slice (4.3–8.6KB/task).
- **MCP now wins every economy metric**: −79% action bytes vs DOM and −42% LLM input tokens. At 30 products it already won bytes (−56%); at 150 the token side crossed too.
- **Both sides 8/8 correct, 16/16 identical picks** — same answers, wildly different cost curves.
- DOM keeps raw-speed lead per task on small lookups but pays 5× the data; MCP time dropped 30→150 (74.3 → 57.3s) since richer filters mean fewer tool round-trips.
- Projection: the gap widens super-linearly for DOM at 500+ products (31KB → ~100KB+ per task), while MCP stays task-bound. Confirms the paper's thesis at real-browser fidelity.

Artifacts: browser_llm.py · results_browser_150.json · catalog.json (150) · demowithmcp / demonomcp.converso.work